

Design Patterns

Mindustry

Realizado por:
Miguel Carmo nº 65871

Design Patterns Identification Report

AI Package - Mindustry Codebase

Location

- Package: *SE2526_65871_67940_68087_68231_68264_68681\core\src\mindustry\ai*

All the locations below this statement are relative locations with the path above

Pattern 1: Strategy Pattern

- Classes:
  - "UnitCommand.java" (lines 14-121)
  - "UnitCommand.controller" field (line 21)
  - "types/mindustry" (various AI controller classes)

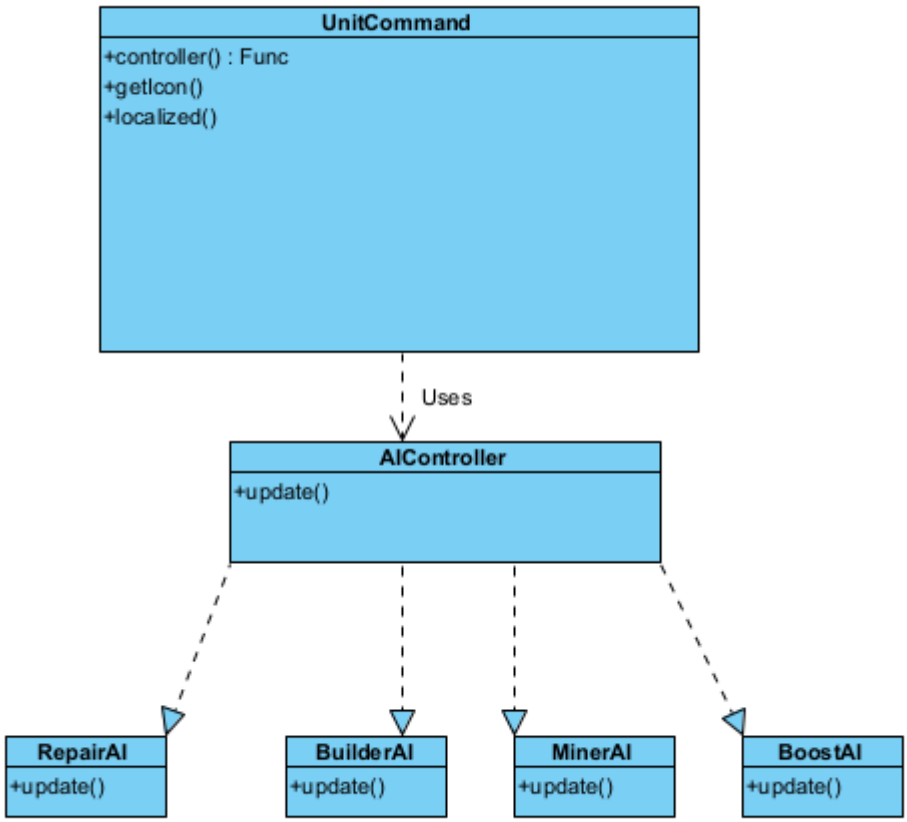
Code Snippet

```
public class UnitCommand extends MappableContent{
    /** Controller that this unit will use when this command is used. */
    public final Func<Unit, AIController> controller;

    public UnitCommand(String name, String icon, Func<Unit, AIController> controller){
        this.controller = controller == null ? u -> null : controller;
    }

    // Strategy usage in loadAll():
    repairCommand = new UnitCommand("repair", "modeSurvival",
        Binding.unitCommandRepair, u -> new RepairAI());
    rebuildCommand = new UnitCommand("rebuild", "hammer",
        Binding.unitCommandRebuild, u -> new BuilderAI());
    mineCommand = new UnitCommand("mine", "production",
        Binding.unitCommandMine, u -> new MinerAI());
}
```

Class Diagram



The Strategy pattern is clearly identified here because:

1. The `UnitCommand` class defines a family of algorithms (different AI behaviors: repair, build, mine, etc.)

2. Strategy Interface: `AIController` serves as the common interface for all strategies

3. Concrete Strategies: `RepairAI`, `BuilderAI`, `MinerAI`, `BoostAI`, etc. are interchangeable algorithms

4. Runtime Selection: The strategy is selected at runtime based on the command assigned to a unit

5. Encapsulation: Each AI behavior is encapsulated in its own class, making them independently testable and maintainable

The pattern allows units to switch between different behaviors dynamically without modifying the unit's core logic, following the Open/Closed Principle

Pattern 2: Strategy Pattern

- Class: `Pathfinder.java`

- Methods:

  - “getField()” (lines 351-363)

  - “fieldTypes” (lines 40-42)

  - “EnemyCoreField” inner class (lines 544-583)

Code Snippet

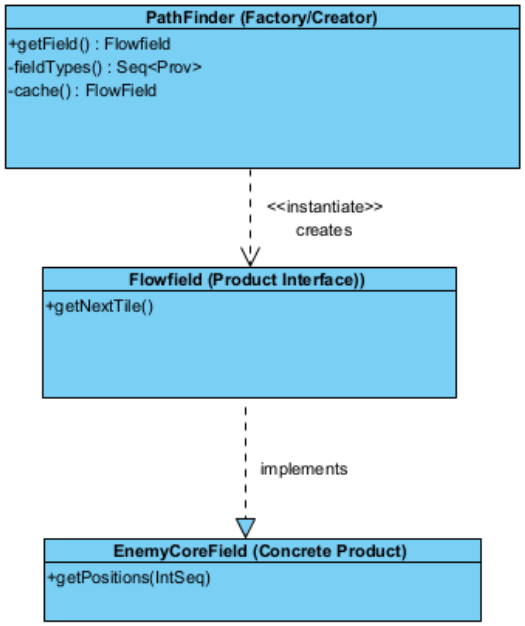
```
public static final Seq<Prov<Flowfield>> fieldTypes = Seq.with(
    EnemyCoreField::new
);

public Flowfield getField(Team team, int costType, int fieldType){
    if(cache[team.id][costType][fieldType] == null){
        Flowfield field = fieldTypes.get(fieldType).get(); // Factory creation
        field.team = team;
        field.cost = costTypes.get(costType);
        field.targets.clear();
        field.getPositions(field.targets);

        cache[team.id][costType][fieldType] = field;
        queue.post(() -> registerPath(field));
    }
    return cache[team.id][costType][fieldType];
}

// Concrete product
public static class EnemyCoreField extends Flowfield{
    @Override
    protected void getPositions(IntSeq out){
        // Implementation specific to enemy core fields
        for(Building other : indexer.getEnemy(team, BlockFlag.core)){
            out.add(other.tile.array());
        }
    }
}
```

Class Diagram



Rationale

The Factory pattern is identified here because:

1. Factory Method: `getField()` acts as the factory method that creates `Flowfield` objects

2. Product Interface: `Flowfield` is an abstract class defining the interface for products

3. Concrete Products: `EnemyCoreField` and `PositionTarget` (line 586) are concrete implementations

4. Dynamic Creation: The factory uses a `Seq<Prov<Flowfield>>` to dynamically create different field types using method references (`EnemyCoreField::new`)

5. Caching: The factory includes caching logic to avoid recreating objects, which is a common factory optimization

6. Encapsulation: Creation logic is encapsulated, and clients don't need to know the specific concrete class

The pattern allows for easy extension with new field types without modifying existing code, following the Open/Closed Principle.

Pattern 3: Strategy Pattern

- Classes:

  - `RtsAI.java` (lines 45-54)

  - `Pathfinder.java` (lines 113-208)

  - `BlockIndexer.java` (lines 56-127)

  - Event system: `mindustry.game.EventType.*`

Code Snippet

```
static{
    Events.on(BuildDamageEvent.class, e -> {
        if(e.build.team.rules().rtsAi){
            var ai = e.build.team.data().rtsAi;
            if(ai != null){
                ai.damagedSet.add(e.build); // React to event
            }
        }
    });
}

// Pathfinder.java - Multiple observers
public Pathfinder(){
    Events.on(WorldLoadEvent.class, event -> {
        stop();
        tiles = new int[world.width() * world.height()];
        // React to world load
    });

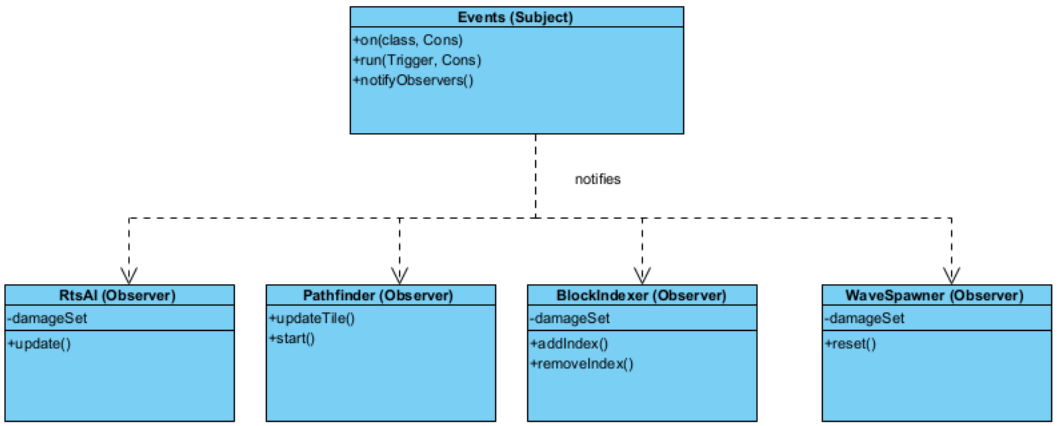
    Events.on(TileChangeEvent.class, event -> {
        if(state.isEditor()) return;
        updateTile(event.tile); // React to tile changes
    });

    Events.run(Trigger.afterGameUpdate, () -> {
        if(needsRefresh && Core.graphics.getFrameId() % 2 == 0){
            // Periodic update observer
        }
    });
}

// BlockIndexer.java - Observer registration
Events.on(TilePreChangeEvent.class, event -> {
    removeIndex(event.tile);
});

Events.on(TileChangeEvent.class, event -> {
    addIndex(event.tile);
});
```

Class Diagram



Rationale

The Observer pattern is clearly identified because:

1. Subject: The `Events` class acts as the subject that maintains a list of observers and notifies them

2. Observers: `RtsAI`, `Pathfinder`, `BlockIndexer`, and `WaveSpawner` are concrete observers

3. Registration: Observers register themselves using `Events.on(EventClass.class, handler)`

4. Notification: When events occur (e.g., `BuildDamageEvent`, `TileChangeEvent`, `WorldLoadEvent`), all registered observers are notified

5. Decoupling: The subject (event system) doesn't know about specific observers, and observers don't know about each other

6. One-to-Many: Multiple observers can react to the same event (e.g., both `Pathfinder` and `BlockIndexer` observe `TileChangeEvent`)

The pattern enables loose coupling between the game event system and AI components, allowing for easy addition of new observers without modifying existing code or the event system itself.

Summary

1. **Strategy Pattern** enables flexible unit behavior switching
2. **Factory Pattern** manages complex object creation for pathfinding
3. **Observer Pattern** maintains loose coupling between game events and AI reactions